PeerLearn: Code Coverage Report

Team: Amogh Shetty (PES1UG24AM034), Aarush Patil (PES1UG24AM005), Abheek Sen (PES1UG24AM009), Allen Anish (PES1UG24AM029). **Course:** Software Engineering, PES University.

Result: 94.9% line coverage, 92.0% statement coverage and 86.9% function coverage across the backend (`backend/src`), from the run of all 61 automated tests on commit `cd6df6f`. The target in NFR-08 is 70% line coverage, enforced in CI. The same run is detailed in the Unit Test Report.

1. Test run

Item	Value
Date of run	31 Oct 2026, 09:58 IST
Code version	`main` branch, commit `cd6df6f` (github.com/amoghshetty05/peerlearn)
Test framework	Jest 29 with Supertest (HTTP API tests)
Environment	Node.js 22, PostgreSQL 16 test database created fresh for each run
Command	`cd backend && npm test` (runs `jest --runInBand --coverage`)
Automation	The same tests run in GitHub Actions (job `backend`) on every pull request and every push to `main`; the job fails if line coverage drops below 70%

2. Summary

Measure	Result
Test suites	8 passed of 8
Test cases	61 passed, 0 failed, 0 skipped
Total test time	16.5 s
Statement coverage	92.0% (837/910)
Branch coverage	80.4% (288/358)
Function coverage	86.9% (205/236)
Line coverage	94.9% (723/762) (target 70%, NFR-08)

3. Code coverage by file

Coverage shows how much of the backend code (`backend/src`, excluding the start-up file `server.js`) the tests executed. The full interactive report is in `docs/reports/coverage/index.html`.

File	Statements	Functions	Lines
src/app.js	100.0%	100.0%	100.0%
src/config.js	80.0%	100.0%	87.5%
src/dal/audit.js	80.0%	50.0%	80.0%
src/dal/groups.js	100.0%	100.0%	100.0%
src/dal/notifications.js	94.7%	88.9%	100.0%
src/dal/pool.js	100.0%	66.7%	100.0%
src/dal/reports.js	100.0%	100.0%	100.0%
src/dal/requests.js	85.7%	66.7%	85.7%
src/dal/sessions.js	81.4%	87.5%	85.4%
src/dal/tutors.js	96.5%	92.8%	96.4%
src/dal/users.js	95.8%	90.9%	100.0%
src/errors.js	100.0%	100.0%	100.0%
src/jobs/scheduler.js	0.0%	0.0%	0.0%
src/middleware/authenticate.js	100.0%	100.0%	100.0%
src/middleware/authorize.js	100.0%	100.0%	100.0%
src/middleware/enforceHttps.js	100.0%	100.0%	100.0%
src/middleware/errorHandler.js	89.5%	100.0%	88.2%
src/middleware/rateLimiter.js	87.1%	75.0%	92.3%
src/middleware/requestId.js	100.0%	100.0%	100.0%
src/middleware/validate.js	96.2%	100.0%	95.2%
src/routes/index.js	92.3%	85.3%	96.9%
src/schemas.js	100.0%	100.0%	100.0%
src/serializers.js	100.0%	100.0%	100.0%
src/services/audit.js	100.0%	100.0%	100.0%
src/services/auth.js	93.2%	100.0%	94.7%
src/services/dashboard.js	100.0%	100.0%	100.0%
src/services/email.js	100.0%	100.0%	100.0%
src/services/group.js	93.1%	100.0%	100.0%
src/services/matching.js	96.4%	85.7%	100.0%
src/services/moderation.js	94.7%	100.0%	100.0%
src/services/notification.js	94.1%	75.0%	100.0%

File	Statements	Functions	Lines
`src/services/profile.js`	85.9%	84.6%	95.5%
`src/services/scoring.js`	100.0%	100.0%	100.0%
`src/services/session.js`	96.0%	85.7%	98.8%
`src/utils/clock.js`	100.0%	100.0%	100.0%
`src/utils/logger.js`	80.0%	75.0%	75.0%
`src/utils/time.js`	100.0%	100.0%	100.0%
Total	**92.0%**	**86.9%**	**94.9%**

4. How to reproduce

1. Start PostgreSQL 16 locally (or use Docker).
2. `cd backend && npm ci`
3. `TEST_ADMIN_URL=postgres://postgres:postgres@localhost:5432/postgres npm test`

In GitHub, open the Actions tab, choose any run of the CI workflow and download the `coverage` artifact for the same HTML coverage report.